

Build in functions

Varying effects

Varying intercepts

```
from BayesForge import bf
import numpy as np

# Setup device-----
m = bf(platform='cpu')

# Import Data & Data Manipulation -----
# Import
from importlib.resources import files
data_path = files('BayesForge.Resources') / 'reedfrogs.csv'
m.data(data_path, sep=';')
# Manipulate
m.df["tank"] = np.arange(m.df.shape[0])

# Define model -----
def model(tank, surv, density):
    alpha = m.effects.varying_intercept(group=tank, group_name = 'tank')
    m.dist.binomial(total_count = density, logits = alpha, obs=surv)

# Run sampler -----
m.fit(model)

# Diagnostic -----
m.summary()
```

Varying slopes

Varying intercepts and slopes

```
from BayesForge import bf
# Setup device-----
m = bf(platform='cpu')

# Import Data & Data Manipulation -----
# Import
from importlib.resources import files
data_path = files('BayesForge.Resources') / 'Sim data multivariatenormal.csv'
m.data(data_path, sep=',')

# Define model -----
def model(cafe, wait, N_cafes, afternoon):
    a = m.dist.normal(5, 2, name = 'a')
    b = m.dist.normal(-1, 0.5, name = 'b')
    sigma = m.dist.exponential(1, name = 'sigma')

    varying_intercept, varying_slope = m.effects.varying_effects(
        N_group = N_cafes,
        group = cafe,
        global_intercept= a,
        global_slope= b,
        group_name = 'cafe')

    mu = varying_intercept + varying_slope* afternoon
    m.dist.normal(mu, sigma, obs=wait)

# Run sampler -----
m.fit(model)
```

Nested varying effects

```
from BayesForge import bf
```

```

# Setup device-----
m = bf(platform='cpu')

# Import Data & Data Manipulation -----
# group_id / region_id are one obs-level index array per level (top to bottom).
# The parent structure (which region each group belongs to) is derived automatically.
data_path = m.load.sim_nested_effects(only_path=True)
m.data(data_path)

# Define model -----
# N_vars = intercept + slopes; it is independent of the number of levels.
def model(y, x, group_id, region_id, N_groups=20, N_regions=5):
    sigma = m.dist.exponential(1)

    a_est, b_est = m.effects.nested_varying_effects(
        N_vars    = 2,                    # intercept + 1 slope
        names     = ["region", "group"],  # levels, top to bottom
        N_groups  = [N_regions, N_groups], # units at each level
        group_ids = [region_id, group_id], # obs-level index per level
        centered  = False,                 # non-centered (recommended)
    )

    mu = a_est + b_est * x
    m.dist.normal(mu, sigma, obs=y)

# Run sampler -----
m.fit(model)
m.summary()

```

`m.effects.nested_varying_effects` generalizes to any depth (add levels to `names`, `N_groups`, `group_ids`) and any number of variables (`N_vars` = intercept + number of slopes, returned as `a_est`, `b1`, `b2`, ...). Pass `centered=True` for the centered parameterization, or supply per-level `sigma` / `L_corr` / `corr` priors to override the defaults. See [Nested varying effects](#) for the full model and math.

Gaussian processes

Squared Exponential Kernel

```
from BayesForge import bf
# Setup device-----
m = bf(platform='cpu')
m.gaussian.kernel_sq_exp
```

Periodic Kernel

```
from BayesForge import bf
# Setup device-----
m = bf(platform='cpu')
m.gaussian.kernel_periodic
```

Locally Periodic Kernel

```
from BayesForge import bf
# Setup device-----
m = bf(platform='cpu')
m.gaussian.kernel_periodic_local
```

Networks effects

Sender receiver

```
sr = m.net.sender_receiver(focal_individual_predictors,target_individual_predictors)
```

Dyadic

```
m.net.dyadic_effect(dyadic_predictors)
```

Block model

```
m.net.block_model(Merica: vector[integer],3)
```

Network metrics

Degree (undirected, out-degree, in-degree)

```
m.net.degree(adj_matrix_jax)
m.net.indegree(adj_matrix_jax)
m.net.outdegree(adj_matrix_jax)
```

Strength (undirected, out-strength , in-strength)

```
m.net.strength(adj_matrix_jax)
m.net.instrength(adj_matrix_jax)
m.net.outstrength(adj_matrix_jax)
```

Eigenvector

```
m.net.eigen(adj_matrix_jax)
```

Clustering coefficient

```
m.net.eigen(adj_matrix_jax)
```

Density

```
m.net.eigen(adj_matrix_jax)
```

Geodesic_distance

```
m.net.eigen(adj_matrix_jax)
```

Diameter

```
m.net.eigen(adj_matrix_jax)
```